



UNIVERSIDADE DA CORUÑA

Facultade de Informática

Máster Universitario en Enxeñaría en Informática

TRABALLO FIN DE MÁSTER

Aplicación para la Gestión de Emergencias

Estudiante: Adrián García Vázquez

Dirección: Manuel Álvarez Díaz

Monforte de Lemos, mayo de 2026.

Dedicatoria

Agradecimientos

Gracias a Manuel, mi tutor, por su paciencia infinita y sus consejos que me han permitido hacer realidad este proyecto.

Gracias a mi familia, mis amigos y a mi novia por el apoyo incondicional y ánimos.

También agradezco a mis compañeros y profesorado de MUEI que me han ayudado a mejorar como informático y como persona.

Resumen

Galicia afronta de forma recurrente emergencias de alto impacto (incendios, inundaciones, derrumbes y riesgos industriales), que exigen una respuesta rápida y coordinada. En este contexto, la gestión operativa se ve limitada por la fragmentación de herramientas, la escasa integración de datos y la necesidad de coordinar a múltiples actores en escenarios de alta presión.

Este Trabajo de Fin de Máster forma parte de un sistema previo centrado en incendios forestales y lo amplía hacia una plataforma de coordinación multi-riesgo para Galicia. La solución se apoya en dos pilares: una plataforma web evolucionada y una aplicación móvil nativa en Kotlin para uso en campo y participación ciudadana. El sistema centraliza alertas y emergencias, integra geolocalización y reportes multimedia, coordina recursos y ofrece visualización cartográfica en tiempo real de eventos y despliegues. Además, incorpora notificaciones push y un histórico para análisis post-emergencia, con indicadores de tiempos de respuesta y recursos utilizados. Con ello, el proyecto mejora la interoperabilidad, optimiza la toma de decisiones y refuerza la capacidad de respuesta en el territorio gallego.

Abstract

Galicia recurrently faces high-impact emergencies (fires, floods, landslides, and industrial hazards) that require fast, coordinated response. In this context, operational management is limited by fragmented tools, weak data integration, and the need to coordinate multiple stakeholders in high-pressure scenarios.

This Master's Thesis extends a previous wildfire-focused system into a multi-risk coordination platform for Galicia. The solution is built on two pillars: an enhanced web platform and a native Kotlin mobile app for field use and citizen participation. The system centralizes alerts and incidents, integrates geolocation and multimedia reporting, supports resource coordination, and provides real-time map visualization of events and deployments. It also includes push notifications and historical records for post-incident analysis, with indicators on response times and resource usage. Overall, the project improves interoperability, supports better decision-making, and strengthens emergency response capacity across the Galician territory.

Palabras clave:

- Galicia
- Gestión de emergencias
- Geolocalización
- Coordinación de personas y recursos
- Aplicación Web
- Java - Spring Boot
- React - Redux
- Aplicación móvil Kotlin

Keywords:

- Galicia
- Emergency management
- Geolocation
- Aplicación Web
- Coordination of people and resources
- Java
- React - Redux
- Kotlin mobile application

Índice general

1	Introducción	1
1.1	Determinación de la situación actual	1
1.2	Alcance y objetivos	2
1.2.1	Objetivos principales	3
1.2.2	Objetivos concretos	3
1.3	Estructura de la memoria	4
2	Base Tecnológica	5
2.1	Tecnologías que se mantienen	5
2.2	Tecnologías eliminadas	6
2.3	Tecnologías nuevas	7
3	Estado del arte	8
3.1	Alternativas ya evaluadas en el TFG	8
3.1.1	Jira	8
3.1.2	Asana	8
3.1.3	Global Forest Watch - Fires	8
3.1.4	InciWeb	9
3.1.5	Google Maps	9
3.1.6	Microsoft Teams	9
3.1.7	Odoo	9
3.1.8	PCAM Gestión	9
3.1.9	XeoCode Lite	9
3.2	Nuevas alternativas del mercado	10
3.2.1	Línea Verde	10
3.2.2	S.O.S Emergencias	11
3.3	Iniciativas públicas recientes	11
3.3.1	Xunta de Galicia: nueva app y ampliación de detección temprana	11

3.4	Conclusión	12
4	Análisis de viabilidad	13
4.1	Viabilidad temporal	13
4.2	Viabilidad tecnológica	13
4.3	Viabilidad económica	14
4.4	Conclusión del análisis de viabilidad	14
5	Introducción al desarrollo realizado	15
5.1	Introducción	15
5.2	Metodología e iteraciones	15
5.3	Arquitectura de la Solución	16
6	Planificación y análisis de costes	18
6.1	Planificación	18
6.2	Sprints	20
6.2.1	Sprint 0: Contextualización	20
6.2.2	Sprint 1: Esqueleto y arquitectura base de aplicación móvil	20
6.2.3	Sprint 2: Actualizar la organización de recursos	20
6.2.4	Sprint 3: Emergencias y asignaciones básicas	21
6.2.5	Sprint 4: Nuevo sistema de logs operativos	21
6.2.6	Sprint 5: Notificaciones en Android y aceptación de asignaciones	21
6.2.7	Sprint 6: Protocolos semiautomáticos (motor de reglas simple)	21
6.2.8	Sprint 7 y 8: Finalización, pruebas y despliegue	21
6.3	Seguimiento	21
6.4	Análisis de costes	22
6.4.1	Estimación del esfuerzo	22
6.4.2	Costes directos de personal	22
6.4.3	Costes indirectos	23
6.4.4	Resumen económico	23
6.4.5	Conclusión del análisis de costes	24
7	Requisitos del sistema	25
7.1	Introducción	25
7.2	Actores	26
7.3	Casos de uso	27
7.4	Modelo de casos de uso	28
7.5	Prototipado de interfaz	28

8	Diseño	29
8.1	Introducción y objetivos	29
8.2	Resumen de patrones usados	29
8.3	Arquitectura general	29
8.4	Subsistema Backend	29
8.4.1	Objetivos	29
8.4.2	Arquitectura	29
8.4.3	Modelo del dominio	30
8.4.4	Capa de acceso a datos	30
8.4.5	Capa de lógica de negocio	31
8.4.6	Capa servicios	31
8.4.7	Otros?	31
8.5	Subsistema Frontend	31
8.5.1	Objetivos	31
8.5.2	Arquitectura	31
8.5.3	Capa de acceso al servicio	32
8.5.4	Capa de componentes de interfaz	32
8.5.5	Otros?	32
9	Implementación	33
9.1	Software requerido	33
9.2	Estructura	33
9.3	Instrucciones de compilación	33
10	Pruebas	34
10.1	Introducción	34
10.2	Pruebas unitarias	34
10.3	Pruebas de integración	34
10.4		35
11	Conclusiones y futuras líneas de trabajo	36
11.1	Conclusiones	36
11.2	Aprendizajes	36
11.3	Futuras líneas de trabajo	36
A	Material adicional	38
A.1	Instalación del Software	38
A.2	Manual de usuario	38

Lista de acrónimos	39
Bibliografía	41

Índice de figuras

3.1	Vista de la app Línea Verde	10
3.2	Vista de la app S.O.S Emergencias	11
5.1	Arquitectura de la solución	16
6.1	Diagrama de Gantt de los sprints	20
7.1	Diagrama de la jerarquía de actores del sistema	27

Índice de tablas

3.1	Tabla de comparativa entre aplicaciones	12
6.1	Planificación del Sprint 3: Emergencias y asignaciones básicas	19
6.2	Desglose de costes estimados del proyecto	23

Introducción

GALICIA presenta una elevada exposición a emergencias en su territorio, entre las que destacan especialmente los incendios forestales, las inundaciones derivadas de episodios de lluvia intensa, los derrumbes de carreteras o terraplenes y determinados riesgos industriales. En los últimos años han surgido emergencias que ilustran su importancia y su impacto operativo: campañas con incendios que han afectado a superficies significativas según la estadística oficial [1], episodios de abril de 2026 que motivaron una decena de incidencias en Ourense y la activación de alertas en las Rías Baixas [2, 3], y movimientos sísmicos locales perceptibles por la población, como el registrado en Ponteceso en 2026 [4]. La complejidad operativa de la respuesta exige coordinación entre múltiples actores, toma de decisiones en ventanas temporales reducidas y disponibilidad de información operativa fiable. Estos retos son tanto tecnológicos como organizativos y normativos, ya que confluyen distintos niveles administrativos y tecnologías

Este [Trabajo Fin de Máster \(TFM\)](#) se apoya y amplía el trabajo previo desarrollado en el [Trabajo Fin de Grado \(TFG\)](#) "Aplicación para coordinación de equipos para la prevención y extinción de incendios forestales" [5], que documenta el diseño e implementación de una aplicación web destinada a centralizar avisos, recursos y despliegues en el contexto de incendios forestales en Galicia. El [TFG](#) aporta decisiones y soluciones de diseño que se toman como punto de partida en este [TFM](#).

1.1 Determinación de la situación actual

La situación actual combina capacidades consolidadas de detección, alerta y despliegue con limitaciones de integración entre sistemas y organizaciones. En el ámbito forestal, existen estadísticas nacionales y autonómicas que permiten caracterizar la magnitud del problema [1]. Galicia dispone además de una estructura operativa asentada en torno a [Axencia Galega de Emerxencias \(112 Galicia\)](#) ([AXEGA](#)) y cuenta con planes de emergencia por tipología de riesgo

a nivel autonómico [6, 7].

Galicia dispone de una organización y planificación detallada para distintas tipologías de riesgo, por ejemplo, [Plan de Prevención y Defensa Contra los Incendios Forestales de Galicia \(PLADIGA\)](#) para riesgos forestales. La Consellería de Presidencia centraliza catálogos, protocolos y guías municipales, la Consellería de Medio Rural publica documentación técnica y memorias sobre defensa del monte, y [AXEGA](#) mantiene boletines e instrumentos operativos. Sin embargo, estos planes y protocolos están dispersos entre distintos organismos y repositorios, y no están unificados en una solución digital compartida y accesible tanto para los equipos de gestión de emergencias como para la ciudadanía común [6, 7, 8].

No obstante, persisten dificultades operativas que limitan la eficacia conjunta. La fragmentación de la información hace que avisos ciudadanos, información meteorológica, mapas de emergencias y estado de recursos residan en sistemas segregados. A ello se suma la heterogeneidad organizativa, ya que organismos y equipos tienen procedimientos y necesidades distintas, desde la coordinación estratégica hasta el mando táctico y la intervención en campo. También destaca la variabilidad del riesgo y la concurrencia de eventos, porque no es realista tratar las emergencias de forma aislada y algunas suelen estar relacionadas con otras, lo que exige un modelo multi-riesgo [3, 7]. Por último, la presión temporal y la trazabilidad obligan a disponer de actualización en tiempo real y de un registro de actuaciones para análisis posteriores.

El análisis realizado en el [TFG](#) muestra que estas limitaciones no son debidas a la ausencia de medios, sino a falta de cohesión operativa digital. Por ello, este [TFM](#) propone una evolución del sistema desarrollado en el [TFG](#) hacia una plataforma de coordinación que mejore la integración, la trazabilidad y el soporte a la decisión en contextos multi-riesgo.

1.2 Alcance y objetivos

El alcance de este [TFM](#) es evolucionar la solución técnica y los modelos operativos desarrollados en el [TFG](#) hacia una plataforma de coordinación de emergencias multi-riesgo para Galicia, con dos componentes principales: una plataforma web para gestión y coordinación, y una aplicación móvil nativa para uso en campo y participación ciudadana. Este [TFM](#) parte explícitamente del trabajo técnico y de campo documentado en el [TFG](#) [5].

La solución propuesta se alinea con el trabajo realizado por protección civil y otros organismos de respuesta a emergencias, priorizando la interoperabilidad entre estos organismos, sin pretender sustituir sistemas institucionales existentes, sino complementarlos aportando una capa unificada de gestión, visualización y soporte a la decisión [9, 10].

1.2.1 Objetivos principales

A continuación se enumeran los principales objetivos que debe cumplir la aplicación a desarrollar en este trabajo:

1. Extender el dominio funcional del TFG hacia un enfoque multi-riesgo (incendios, inundaciones, derrumbes y riesgos industriales) acorde con los planes autonómicos.
2. Ampliar funcionalidades de la plataforma web existente que centraliza la gestión de alertas, emergencias, recursos y despliegues operativos.
3. Diseñar e implementar una aplicación móvil nativa en Kotlin para reporte ciudadano y soporte a la intervención en campo.
4. Mejorar la coordinación multi-organizativa, integrando en un mismo flujo de trabajo a coordinadores, brigadas, protección civil y ciudadanía.
5. Reforzar la trazabilidad y el análisis post-emergencia mediante históricos e indicadores operativos.

1.2.2 Objetivos concretos

1. Contextualización: esclarecer el valor aportado por el TFM y definir el nuevo dominio multi-riesgo.
2. Definir la arquitectura de la aplicación móvil y entregar un esqueleto que permita añadir funcionalidades progresivamente.
3. Introducir un modelo unificado de recursos y organizaciones y definir sus capacidades y área operativa.
4. Permitir la creación y edición de emergencias (por punto o por cuadrante) y la creación o propuesta manual de asignaciones.
5. Introducir un registro de logs operativos de las emergencias y de las asignaciones de recursos que permita el análisis posterior de las emergencias y la generación de indicadores.
6. Integrar notificaciones móviles mediante [Firebase Cloud Messaging \(FCM\)](#), registrar dispositivos y definir el flujo de notificación y aceptación de asignaciones.
7. Implementar el almacenamiento de reglas basándose en los protocolos actuales y un motor de reglas que sea capaz de generar propuestas de asignación automáticamente.

1.3 Estructura de la memoria

La memoria se organiza en once capítulos y anexos. A continuación se ofrece un resumen del contenido de cada capítulo:

1. Capítulo 1: Presenta el contexto general, describe la situación actual en Galicia, expone las motivaciones del trabajo y enuncia los objetivos generales y concretos del proyecto.
2. Capítulo 2: Detalla la base tecnológica empleada: componentes de backend y frontend, bases de datos, servicios y librerías relevantes, y las decisiones arquitectónicas que sustentan la implementación.
3. Capítulo 3: Revisa el estado del arte, compara soluciones existentes y extrae lecciones aplicables al diseño de la plataforma propuesta.
4. Capítulo 4: Analiza la viabilidad del proyecto desde las perspectivas técnica, organizativa y normativa, e incluye una estimación de recursos y riesgos.
5. Capítulo 5: Documenta el desarrollo realizado, describiendo la arquitectura global del sistema, los módulos principales y las integraciones implementadas.
6. Capítulo 6: Presenta la planificación temporal del trabajo, los hitos y la metodología seguida para organizar las iteraciones de desarrollo.
7. Capítulo 7: Especifica los requisitos funcionales y no funcionales que guían el diseño y la implementación de la plataforma.
8. Capítulo 8: Describe el diseño funcional y técnico, incluyendo el modelo de datos, las APIs, los flujos de interacción y las decisiones de UX/UI.
9. Capítulo 9: Detalla la implementación: estructura de código, despliegue, configuraciones y ejemplos de componentes desarrollados.
10. Capítulo 10: Documenta las pruebas realizadas (unitarias, de integración y de aceptación), su cobertura y los resultados obtenidos.
11. Capítulo 11: Resume las conclusiones, las aportaciones del trabajo y propone líneas futuras de mejora y continuidad.

Base Tecnológica

A continuación se presenta la base tecnológica del TFM, organizada para distinguir entre las tecnologías que se mantienen respecto al TFG, las que se han eliminado y las que se han incorporado como novedades.

2.1 Tecnologías que se mantienen

Estas tecnologías ya estaban presentes en el TFG y se conservan porque siguen siendo adecuadas para el sistema, aportan estabilidad y permiten mantener continuidad técnica.

- **Java** [11] sigue siendo el lenguaje principal del backend por su madurez y su integración natural con **Spring** (Spring).
- **Structured Query Language (SQL)** [12] se mantiene como base para definir y consultar los datos persistentes.
- **HyperText Markup Language (HTML)** y **Cascading Style Sheet (CSS)** [13] continúan siendo la base de la estructura y presentación de la interfaz web.
- **JavaScript** (JavaScript) y **JavaScript XML (JSX)** [14, 15] se conservan en la capa frontend por su papel en la lógica de interacción y en la construcción de componentes **React**.
- **LaTeX** (LaTeX) [16] se mantiene para la redacción y composición de la memoria.
- **JavaScript Object Notation (JSON)** [17] sigue siendo el formato de intercambio de datos entre cliente y servidor.
- **Spring** (Spring) [18], **Spring Boot** (Spring Boot) [19], **Spring Data** (Spring Data) [20], **Java Persistence API (JPA)** [21], **Hibernate** [22] y **Java Database Connectivity (JDBC)** [23] permanecen como núcleo del backend por su robustez y por facilitar el acceso a datos y la lógica de negocio.

- [React \(React\)](#) [24], [Redux \(Redux\)](#) [25], [Redux Toolkit Query \(RTK Query\)](#) [26] y [Material UI](#) [27] se conservan como base de la interfaz web por su capacidad para construir una aplicación modular y mantenible.
- [Open Weather Map](#) [28] se mantiene como fuente externa de información meteorológica útil en el contexto de emergencias.
- [MapLibre](#) [29] sigue siendo la solución cartográfica abierta utilizada para visualizar información geoespacial.
- [JUnit \(JUnit\)](#) [30] se conserva como herramienta principal de pruebas unitarias.
- [Hypertext Transfer Protocol \(HTTP\)](#) y [Hypertext Transfer Protocol Secure \(HTTPS\)](#) [31] y [Representational State Transfer \(REST\)](#) [32] continúan siendo la base de la comunicación cliente-servidor.
- [Integrated Development Environment \(IDE\)](#) [IntelliJ](#) [33], [Postman](#) [34], [VSCode](#) [35], [DBaver](#) [36], [Git](#) [37], [Apache Maven](#) [38], [Create React App](#) [39], [PostgreSQL](#) [40], [PostGIS](#) [41], [Tomcat](#) [42], [Webpack](#) [43], [Serve](#) [44] y [Docker](#) [45] se mantienen como herramientas y servicios de desarrollo y despliegue porque siguen resolviendo correctamente las necesidades del proyecto.

2.2 Tecnologías eliminadas

Estas tecnologías estaban presentes en el TFG, pero en el TFM han sido sustituidas por alternativas más adecuadas.

- [Liquibase](#) [46] y [H2](#) [47] se han dejado de usar como combinación principal porque en este TFM la base de datos real se despliega mediante [Docker](#) [45] sobre [PostgreSQL](#) [40] y [PostGIS](#) [41]. De este modo, el entorno de desarrollo y el de ejecución se acercan más al entorno final y se evita mantener una base de datos en memoria con una configuración distinta a la de producción. Además, la estrategia de pruebas se apoya en [Testcontainers](#) [48], que permite levantar una instancia aislada de la base de datos durante el testing y reproducir con más fidelidad el comportamiento real del sistema.
- [React Map GL](#) [49] se elimina como dependencia explícita porque la integración cartográfica se centraliza sobre [MapLibre](#) [29].
- [Mapbox](#) [50] se descarta como solución principal para reducir la dependencia claves API junto con planes de prueba/gratuitos.

2.3 Tecnologías nuevas

Estas tecnologías se incorporan específicamente en el TFM para cubrir nuevas necesidades, sobre todo en la aplicación Android y en la organización del proyecto.

- Kotlin [51] se añade como lenguaje principal de la aplicación Android por su concisión e interoperabilidad con Java.
- Gradle [52] se incorpora como sistema de construcción habitual en el ecosistema Android.
- [Jetpack Compose \(Compose\)](#) [53] se añade para crear interfaces Android de forma declarativa y más mantenible.
- [Material Design \(Material\)](#) [54] se incorpora como guía visual para la app móvil.
- [HttpClient](#) [55] se añade como biblioteca para realizar peticiones HTTP desde Android.
- [Firebase Cloud Messaging \(FCM\)](#) [56] se incorpora para el envío de notificaciones push.
- [FirebaseMessagingService \(FMS\)](#) [57] se añade para gestionar la recepción de mensajes procedentes de FCM.
- [Testcontainers \(Testcontainers\)](#) [48] se incorpora para ejecutar pruebas de integración sobre una base de datos aislada y reproducible. Esta tecnología está basada en Docker [45] y permite levantar contenedores específicos para cada prueba o conjunto de pruebas, garantizando un entorno controlado y evitando interferencias entre pruebas.

Estado del arte

EN ESTE CAPÍTULO se realiza un análisis de las aplicaciones y herramientas existentes que pueden aportar soluciones para abordar la problemática comentada en este [TFM](#).

El objetivo no es ofrecer una revisión exhaustiva, sino separar de forma clara las alternativas ya citadas en el [TFG](#) de las incorporaciones nuevas, manteniendo en ambos casos una evaluación muy breve de su utilidad para la coordinación de emergencias.

3.1 Alternativas ya evaluadas en el TFG

Estas alternativas ya fueron analizadas en el [TFG](#), a continuación se ofrece un resumen de cada una de ellas, con especial atención a sus fortalezas y limitaciones en el contexto de coordinación de emergencias.

3.1.1 Jira

Jira [\[58\]](#) es una plataforma de gestión de incidencias y proyectos útil para crear, priorizar y asignar tickets. Sin embargo, no aporta representación geoespacial nativa, por lo que su encaje en emergencias es parcial.

3.1.2 Asana

Asana [\[59\]](#) ofrece gestión de tareas, prioridades y colaboración en equipo. Su principal carencia para este caso es que no integra mapas ni soporte geográfico.

3.1.3 Global Forest Watch - Fires

Global Forest Watch Fires [\[60\]](#) aporta cartografía y alertas sobre incendios a partir de datos satelitales, pero no está pensada para coordinar recursos humanos ni para participación ciudadana.

3.1.4 InciWeb

InciWeb [61] centraliza información sobre incidentes y grandes incendios con mapas e informes, pero no incorpora avisos colaborativos ni gestión táctica de equipos.

3.1.5 Google Maps

Google Maps [62] permite mostrar alertas geoespaciales y ofrece una cartografía muy amplia, aunque no resuelve por sí mismo la gestión de despliegue ni el flujo de incidentes.

3.1.6 Microsoft Teams

Microsoft Teams [63] ayuda a organizar equipos y comunicación, pero no ofrece geolocalización ni procesos específicos para emergencias.

3.1.7 Odoo

Odoo [64] es flexible y modular, pero requiere personalización para ajustarse a flujos de coordinación de emergencias y a soporte geográfico.

3.1.8 PCAM Gestión

PCAM Gestión [65] ofrece avisos, asignación de personal y notificaciones en el ámbito de la protección civil, aunque con capacidades geoespaciales limitadas.

3.1.9 XeoCode Lite

XeoCode Lite [66] es una herramienta interna y privada de la Xunta de Galicia. Permite consultar el origen y evolución de incendios, recursos asignados y cartografía asociada, pero no expone información funcional al público ni permite participación ciudadana directa.

3.2 Nuevas alternativas del mercado

3.2.1 Línea Verde



Figura 3.1: Vista de la app Línea Verde

Línea Verde [67] es una plataforma municipal de participación ciudadana y gestión de incidencias. Está bien orientada a comunicación con la administración, pero su foco es urbano y no operativo para emergencias.

3.2.2 S.O.S Emergencias

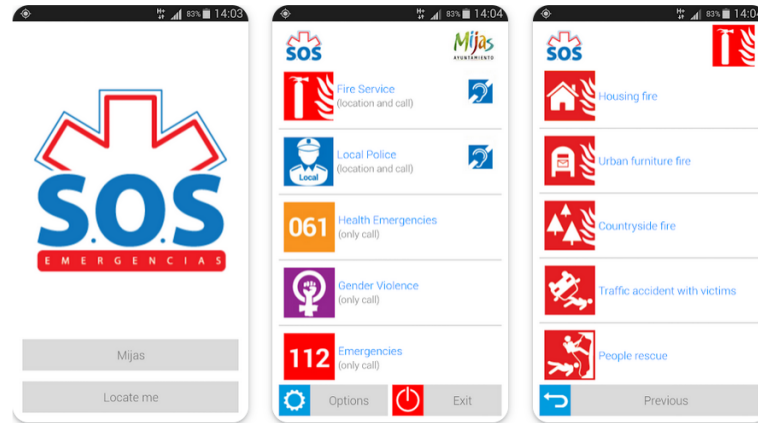


Figura 3.2: Vista de la app S.O.S Emergencias

S.O.S Emergencias [68] es una aplicación móvil orientada a la notificación y gestión básica de emergencias. Aporta una aproximación más directa al ámbito de alertas, aunque no integra una visión completa de coordinación geoespacial y despliegue de recursos.

3.3 Iniciativas públicas recientes

3.3.1 Xunta de Galicia: nueva app y ampliación de detección temprana

En una comunicado, la Xunta de Galicia ha anunciado la ampliación de mecanismos de detección temprana de incendios forestales mediante la creación de una nueva app para la ciudadanía junto el uso de cámaras de videovigilancia.[69].

Esta aplicación combina un canal para notificaciones tempranas, sensores y cámaras para observación remota y procesamiento automatizado con IA para priorizar alertas y detectar anomalías. Todo ello para ayudar a la notificación y coordinación de emergencias en tiempo real.

Funcionalmente esta nueva app parece converger mucho con el dominio funcional del TFG [5]: reportes ciudadanos, integración de sistemas de información dispares como avisos ciudadanos y meteorológicos y también cuenta con representación geoespacial; a efectos prácticos la solución de la Xunta incorpora muchas de las ideas que se prototiparon en el TFG.

Algunas diferencias clave respecto al TFG son que la aplicación añade una capa de hardware y vigilancia (cámaras/sensores) y está diseñada para integrarse en procedimientos administrativos y operativos a escala autonómica. Por otro lado, el TFG aporta un énfasis en

la coordinación táctica como la asignación de recursos, gestión por cuadrantes y soporte a decisiones de despliegue en campo.

Como comentario crítico, cabe destacar la coincidencia temática que resalta la relevancia del problema abordado en el TFG y demuestra que las soluciones propuestas tienen aplicación real. Ambas aproximaciones son complementarias y evidencian la necesidad de soluciones integrales que aborden todo el ciclo de gestión de emergencias, desde la detección hasta la coordinación en campo.

3.4 Conclusión

Las aplicaciones analizadas ofrecen soluciones parciales en las tres dimensiones de interés: gestión de avisos, representación geoespacial y coordinación de equipos. Ninguna de las herramientas revisadas integra de forma nativa las tres capacidades con enfoque ciudadano-operativo y despliegue de recursos; esto evidencia el hueco funcional que aborda la propuesta de este trabajo.

La tabla comparativa 3.1 resume las funcionalidades analizadas para resaltar fortalezas y carencias de cada alternativa:

	Visualización geográfica	Datos meteorológicos	Gestión de personal y recursos	Plan gratuito suficiente	Plugins o margen de mejora	Sistema de avisos o notificaciones	Multi-emergencia	App móvil
Jira			✓	✓	✓	✓	✓	✓
Asana			✓	✓	✓	✓	✓	✓
Línea Verde	✓		✓	✓		✓	✓	✓
SOS Emergencias			✓	✓		✓	✓	✓
Global Forest Watch - Fires	✓	✓		✓		✓		
InciWeb	✓			✓				
Google Maps	✓			✓	✓			✓
Microsoft Teams			✓	✓	✓	✓	✓	✓
Odoo			✓	✓	✓	✓	✓	✓
PCAM Gestión	✓		✓		✓	✓	✓	✓
XeoCode Lite	✓	?	✓	?	?	?	?	?
Xunta (nueva app)	✓	✓		✓		✓		✓

Tabla 3.1: Tabla de comparativa entre aplicaciones

Análisis de viabilidad

EN ESTE CAPÍTULO se analiza la viabilidad del proyecto desde tres perspectivas complementarias: dimensión temporal, dimensión tecnológica y dimensión económica.

4.1 Viabilidad temporal

Para la realización de este TFM se ha estimado una duración aproximada de cuatro meses. Dicha planificación incluye las fases de análisis, diseño, implementación, pruebas y redacción de la memoria. Además, se contempla la familiarización de las nuevas tecnologías. Dado el alcance definido la viabilidad temporal es aceptable y no representa un riesgo crítico para la finalización del proyecto.

La metodología usada utiliza desarrollos cortos para ir validando el progreso y ajustando el alcance si fuera necesario.

4.2 Viabilidad tecnológica

Para el estudio de viabilidad tecnológica se han considerado las tecnologías heredadas del TFG, así como las nuevas tecnologías propuestas para el desarrollo de la aplicación móvil y la integración de notificaciones push.

En resumen, el stack tecnológico propuesto para el TFM es el siguiente:

- Backend: Spring Boot (Java) para servicios REST y la integración con PostgreSQL/PostGIS.
- Base de datos: PostgreSQL con PostGIS para soporte espacial.
- Frontend web: React con Material-UI para interfaces ricas.
- Aplicación Android: Kotlin y Jetpack Compose para la capa de presentación; cliente nativo para acceder a sensores y recibir notificaciones push.

- Notificaciones push: Firebase Cloud Messaging (FCM) para la entrega fiable de mensajes a dispositivos Android.

En lo relativo de Kotlin y FCM, ambas tecnologías están ampliamente adoptadas, cuentan con extensa documentación, y existen guías oficiales para su integración con backends Java/Spring. Por tanto, su adopción no supone un riesgo tecnológico significativo, aportan ventajas en productividad, interoperabilidad móvil y soporte nativo para notificaciones.

Las demás tecnologías son conocidas por el autor gracias a su dedicación profesional y al proyecto previo, lo que reduce el riesgo de adopción y acelera el desarrollo.

4.3 Viabilidad económica

El proyecto no requiere una inversión económica significativa en hardware o licencias de software. El software utilizado es mayoritariamente Open Source (Spring Boot, PostgreSQL, React, Kotlin/Jetpack Compose), por lo que no hay costes de licencia en el entorno de desarrollo académico.

En el ámbito del desarrollo y las pruebas estos pueden ejecutarse con recursos disponibles por lo que no es necesario adquirir nuevo hardware. Sobre el uso de servicios en la nube o APIs de terceros (FCM, MapLibre, OpenWeatherMap) pueden surgir costes, pero se pueden controlar gracias a los planes gratuitos o a la monitorización de uso.

La dimensión económica no supone un riesgo significativo para la ejecución del TFM pues los costes directos son bajos.

4.4 Conclusión del análisis de viabilidad

En resumen, el proyecto es totalmente viable porque combina un plan de trabajo realista de cuatro meses con un uso inteligente de la tecnología. Al reutilizar la base del TFG y apoyarse en herramientas gratuitas y potentes como Kotlin y FCM, el riesgo técnico es mínimo y el coste es prácticamente cero. Esta experiencia previa, sumada a una planificación clara, garantiza que el proyecto se pueda terminar a tiempo y con la calidad necesaria para un TFM.

Introducción al desarrollo realizado

5.1 Introducción

EN este capítulo se presenta una descripción del desarrollo realizado, los objetivos perseguidos y la metodología aplicada durante la implementación. El propósito es ofrecer un vistazo general de los artefactos construidos y de las decisiones más relevantes tomadas durante el desarrollo del proyecto.

5.2 Metodología e iteraciones

El proceso de desarrollo se organizó mediante iteraciones cortas con objetivos funcionales concretos, siguiendo las ideas de Scrum adaptadas al contexto de trabajo individual. Aunque no se aplicó una implementación completa y formal de Scrum (debido al tamaño reducido del equipo), se adoptaron sus pilares fundamentales: planificación por sprints, revisiones y *feedback* continuo.

En cuanto a roles, el autor del TFM asumió las responsabilidades de desarrollo y encargado de definición funcional, mientras que el tutor actuó como responsable de supervisión y de aceptación.

Se establecieron reuniones de revisión mensuales con el tutor en las que se presentaron los artefactos construidos (prototipos, endpoints, pantallas, pruebas y documentación). Estas sesiones sirvieron para validar funcionalidades, priorizar tareas y ajustar el dominio del TFM y el alcance de las siguientes iteraciones.

Para la planificación y el detalle de cada sprint se describen con mayor extensión en el capítulo de planificación (capítulo 6.2).

complementa la visualización con MapLibre y el consumo de información meteorológica de OpenWeatherMap.

Como ampliación respecto al TFG, el TFM incorpora una aplicación Android desarrollada en Kotlin y Jetpack Compose, que reutiliza la misma API para consultar emergencias, asignaciones y recursos. Además, la mensajería con Firebase Cloud Messaging permite que el backend notifique eventos relevantes a los dispositivos móviles, cerrando el flujo de comunicación entre la capa de gestión y la app de campo.

Planificación y análisis de costes

EN este capítulo se presenta la planificación aplicada durante el desarrollo del proyecto y un análisis de los costes asociados. Se describe la planificación inicial, las sprints que han estructurado el trabajo y las desviaciones observadas en el seguimiento.

6.1 Planificación

La planificación inicial se definió en 9 sprints, cada una organizada internamente siguiendo el modelo clásico de cuatro fases: análisis, diseño, implementación y pruebas. En este trabajo se considera que cada sprint equivale a un bloque de dos semanas, lo que facilita agrupar el esfuerzo y revisar avances de forma periódica. La estimación de esfuerzo inicial se basó en jornadas de 6 horas diarias y en una duración prevista del proyecto de cuatro meses. No obstante, las complejidades técnicas y errores imprevistos motivaron desviaciones en varias iteraciones, tal y como se comenta en la sección de seguimiento.

Tarea	Fecha inicio	Días estimados	Días reales
Definición tipos Emergency	04/03	0.5	0.5
Definición modelo Assignment	04/03	0.5	0.5
Definición contratos OpenApi	04/03	1	1
Diseño entidades JPA	05/03	0.5	0.5
Implementación entidad Emergency	06/03	0.5	0.5
Implementación entidad Assignment	06/03	0.5	0.5
Implementación repositorios	06/03	0.5	0.5
UseCase Emergency	08/03	0.5	0.5
UseCase Assignment	08/03	0.5	0.5
Endpoints /emergencies	09/03	0.5	0.5
Endpoints /assignments	09/03	0.5	0.5
Tests unitarios backend	11/03	1	1
Tests integración backend	11/03	1	1
Consumir Api desde frontend	10/03	0.5	0.5
Vista mapa (crear Emergency)	11/03	1	1
Formulario Emergency	11/03	0.5	0.5
Listado Emergencies	12/03	0.5	0.5
Actualización UI Assignment	12/03	1	1
Listado Assignments	13/03	0.5	0.5
Consumir Api desde Android	07/03	0.5	0.5
Lista Assignments Android	12/03	1	1
Detalle Assignment Android	13/03	0.5	0.5
Corrección bugs	15/03	1	1

Tabla 6.1: Planificación del Sprint 3: Emergencias y asignaciones básicas

En la tabla 6.1 se puede observar un desglose del sprint 3, que se centró en la implementación de la gestión de emergencias y asignaciones básicas. Cada tarea se estimó inicialmente en función de su complejidad, esto sirvió para poder observar desviaciones y ajustar la planificación de los siguientes sprints. En este caso, el sprint se completó dentro del plazo previsto, lo que permitió mantener el ritmo de desarrollo planificado.

6.2 Sprints

El proyecto se estructuró en una sucesión de sprints, con una duración de dos semanas para cada uno de los cuales incrementaba la funcionalidad de los artefactos entregados. A continuación se detalla cada sprint con sus objetivos y resultados principales.

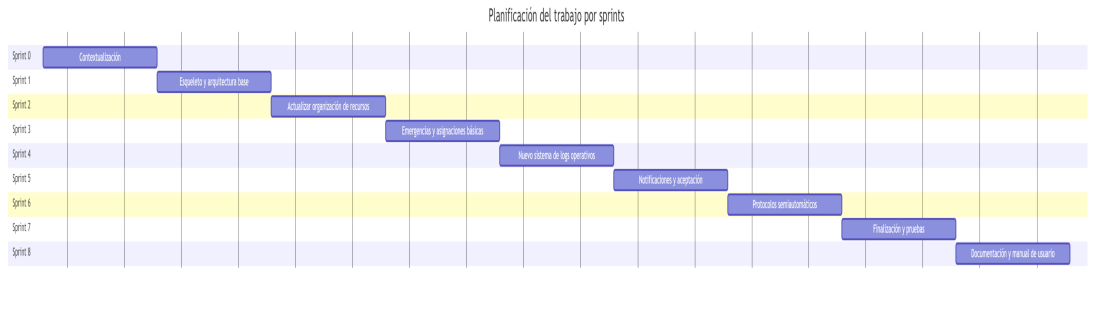


Figura 6.1: Diagrama de Gantt de los sprints

6.2.1 Sprint 0: Contextualización

En esta fase inicial se realizó la contextualización del dominio y la justificación del valor del TFM. Se investigaron fuentes, alternativas a la aplicación y tecnologías potenciales (mapas, sistema de notificaciones android, sistema de recomendación y reglas básico). El objetivo fue delimitar el alcance y dominio del proyecto para que este sea viable en un espacio temporal acotado.

6.2.2 Sprint 1: Esqueleto y arquitectura base de aplicación móvil

Se creó el esqueleto del proyecto móvil y se definieron las dependencias y la arquitectura base (estructura de paquetes, patrones de navegación, y la configuración inicial de Jetpack Compose). Además se implementaron los módulos iniciales de autenticación y sincronización con el Backend (endpoints básicos) para poder iterar sobre funcionalidades reales.

6.2.3 Sprint 2: Actualizar la organización de recursos

Se introdujo el modelo unificado de recursos y organizaciones preservando la compatibilidad con las entidades Legacy (Equipo y Vehículo). Esto incluyó definir atributos de capacidad, áreas operativas (cuadrantes) y las relaciones necesarias a nivel de base de datos y DTOs. Se arreglaron los endpoints antiguos y DTO's para la gestión CRUD de organizaciones y sus recursos.

6.2.4 Sprint 3: Emergencias y asignaciones básicas

Se implementó la gestión de emergencias con soporte para dos tipos de localización: punto y cuadrante. Se añadió la creación y edición de Emergencias, la definición de cuadrantes y una primera versión del flujo de creación de Asignaciones. Esta iteración incluyó validaciones de consistencia (ej. comprobar que un cuadrante pertenece a una emergencia) y pruebas de integración explícitas para el servicio de Asignaciones.

6.2.5 Sprint 4: Nuevo sistema de logs operativos

Se definió un nuevo esquema para los logs operativos, ahora centralizados en las Asignaciones de recursos en las emergencias. El trabajo abarcó el nuevo modelado y la persistencia que permiten conservar el histórico.

6.2.6 Sprint 5: Notificaciones en Android y aceptación de asignaciones

Se integró Firebase Cloud Messaging (FCM) en la aplicación Android: registro de MobileDevice, manejo de tokens y recepción de notificaciones. Se implementó el flujo de envío de notificaciones desde el Backend y la experiencia de usuario en la app para aceptar o rechazar asignaciones recibidas.

6.2.7 Sprint 6: Protocolos semiautomáticos (motor de reglas simple)

Se introdujo la capacidad de definir Protocolos almacenados en JSONB y un motor de reglas ligero que permite generar propuestas de Asignaciones de forma semiautomática en el frontend. Esto incluyó la definición del formato JSON para los protocolos, la implementación de un evaluador simple y la integración con el pipeline de creación de propuestas.

6.2.8 Sprint 7 y 8: Finalización, pruebas y despliegue

Las iteraciones finales se dedicaron a la estabilización: pruebas end-to-end, ajuste de UX tanto del frontal como de la aplicación Android, corrección de errores, poblamiento de datos de ejemplo y preparación de artefactos de despliegue (contenedores Docker para BBDD y servicio). También incluyeron la redacción y revisión de la memoria y la generación de la documentación.

6.3 Seguimiento

El seguimiento se llevó a cabo mediante reuniones mensuales con el tutor y el uso de un tablero Kanban de tareas que permitía visualizar el estado de ellas. En cada revisión se presentaron artefactos funcionales (pantallas, endpoints, wireframes).

Las desviaciones más relevantes provinieron de problemas técnicos con la representación de cuadrantes en el mapa, de la configuración de la aplicación Android y, especialmente, de la integración del backend con Firebase Cloud Messaging (FCM) y del motor de reglas básico para los protocolos. En el primer caso fue necesario revisar documentación oficial y ejemplos de proyectos similares para ajustar correctamente la configuración de notificaciones y el envío de mensajes. En el segundo, al tratarse de una tecnología y un enfoque de diseño que el autor no había implementado previamente, se dedicó tiempo adicional a comprender el patrón y a validar su encaje en el flujo de propuestas, lo que retrasó parcialmente algunas entregas.

6.4 Análisis de costes

El análisis de costes se ha planteado distinguiendo entre costes directos de personal, costes indirectos de infraestructura y un margen de contingencia para imprevistos. Para la valoración económica del esfuerzo se han tomado como referencia salarios publicados por Talent.com para España, donde un perfil de *Programador* se sitúa en 27.500 €/año (14,10 €/h) [70] y un perfil de *Analista programador* en 29.120 €/año (14,93 €/h) [71]. Estos valores resultan razonables para un TFM técnico con tareas de análisis, desarrollo, pruebas e integración.

6.4.1 Estimación del esfuerzo

La planificación se ha estructurado en 7 sprints. Si se asume una duración media de 2 semanas por sprint, el proyecto cubre aproximadamente 14 semanas de trabajo efectivo. Considerando una dedicación máxima de 6 horas al día y trabajo únicamente de lunes a viernes, el esfuerzo total estimado se calcula como sigue:

$$7 \text{ sprints} \times 2 \text{ semanas/sprint} \times 5 \text{ días/semana} \times 6 \text{ h/día} = 420 \text{ horas}$$

Este valor representa una estimación prudente del esfuerzo total del proyecto. La distribución del tiempo no es homogénea entre sprints, ya que las primeras iteraciones requieren más análisis y diseño, mientras que las finales concentran más trabajo de integración, pruebas, corrección de errores y documentación.

6.4.2 Costes directos de personal

Para la valoración del trabajo se ha tomado como base el perfil de *Analista programador*, al ser el que mejor refleja un proyecto de estas características, donde una misma persona asume tareas de definición funcional, implementación y validación. Con la tarifa media indicada por la fuente utilizada, el coste directo estimado es:

$$420 \text{ h} \times 14,93 \text{ €/h} = 6.270,60 \text{ €}$$

De forma complementaria, si se valorase el trabajo con el perfil de *Programador*, el coste sería:

$$420 \text{ h} \times 14,10 \text{ €/h} = 5.922,00 \text{ €}$$

Para reflejar ambas referencias, se adopta como coste base del proyecto el punto medio entre ambos perfiles, es decir, **6.096,30 €**, que representa mejor la mezcla de tareas de programación y análisis realizada durante el desarrollo.

6.4.3 Costes indirectos

A los costes de personal se añaden los costes indirectos de ejecución. Para este tipo de proyecto, los más relevantes son la amortización del equipo de trabajo y los suministros básicos asociados al desarrollo:

- Amortización de hardware: se considera el uso de un equipo portátil de desarrollo valorado en 900 €, con una vida útil de 4 años. Para un proyecto de unos 3,5 meses, el coste imputable es de aproximadamente 66,00 €.
- Conectividad y suministros: se estiman 40 €/mes de internet y 30 €/mes de electricidad proporcional al tiempo de desarrollo. Para 3,5 meses, esto supone 245,00 €.

6.4.4 Resumen económico

Concepto	Importe estimado (€)	Observaciones
Coste directo de personal	6.096,30	Media entre programador y analista programador
Amortización de hardware	66,00	Portátil de desarrollo imputado al proyecto
Conectividad y suministros	245,00	Internet y electricidad proporcionales
Total estimado	6.407,3	

Tabla 6.2: Desglose de costes estimados del proyecto

La tabla 6.2 muestra que el mayor peso del presupuesto corresponde al trabajo humano, seguido por los costes de infraestructura y suministros. Esto es coherente con un proyecto de software de ámbito académico, donde la inversión principal no está en equipamiento especializado sino en la dedicación de horas de desarrollo, integración y validación.

6.4.5 Conclusión del análisis de costes

El coste final del proyecto asciende a **6.407,3€** con una dedicación total de **420 horas**. Estas cifras reflejan el alcance y la coste del **TFM**, considerando tanto el esfuerzo humano como los recursos materiales necesarios para su ejecución. Es importante destacar que esta estimación se ha realizado con criterios prudentes y basándose en datos salariales reales, lo que aporta una visión más ajustada del valor económico del trabajo realizado en este proyecto.

Requisitos del sistema

EN este capítulo se describen los requisitos funcionales y los actores principales del sistema. La solución evoluciona respecto al TFG ampliando el alcance desde la coordinación centrada en incendios forestales hacia una plataforma multi-riesgo, con un flujo más completo de aviso, validación, creación de emergencias y asignación de recursos.

7.1 Introducción

Los tres pilares funcionales de la aplicación siguen siendo la gestión de avisos, la gestión de emergencias y la gestión de personal y recursos, pero en este TFM se redefinen para cubrir un dominio más amplio. Los usuarios no autenticados o ciudadanos pueden registrar avisos sobre emergencias con sus imágenes adjuntas y consultar posteriormente el estado de dichos avisos.

Sobre esos avisos actúa un administrador o coordinador, que revisa la información recibida y decide si procede crear una emergencia. A diferencia del TFG, donde el foco estaba más ligado al incendio forestal y a una organización más estática de equipos, en este trabajo la emergencia puede vincularse a unas coordenadas en concreto, cuando el incidente está localizado con precisión, o como una colección de cuadrantes, cuando este afecta a una zona más amplia.

Una vez creada la emergencia, el administrador puede seguir los protocolos estipulados y realizar una asignación manual de los recursos, pero ahora el sistema ofrece una propuesta de asignación de recursos a dicho coordinador. Este mecanismo filtra candidatos en función de la organización a la que pertenecen, de su estado operativo y de su distancia a la emergencia.

El flujo operativo se completa con el envío de notificaciones a los usuarios de los equipos asignados, de forma que el jefe de equipo pueda aceptar la emergencia y se actualice el estado del recurso a ocupado o desplegado. En el caso de los vehículos, la aceptación se resuelve de forma automática. Finalmente, cuando la emergencia se completa, el recurso pasa de nuevo a

estado disponible.

La principal ampliación funcional respecto al TFG es la introducción de un rol adicional de miembro de equipo. Así, el sistema distingue ahora entre ciudadano, miembro de equipo, manager y coordinador. El ciudadano conserva un perfil muy reducido, centrado en crear avisos y consultar el estado de sus notificaciones. El miembro de equipo accede al seguimiento de sus asignaciones, el manager mantiene funciones de organización y gestión operativa, y el coordinador conserva las capacidades de validación, creación de emergencias y supervisión global del sistema.

7.2 Actores

En el contexto de una plataforma de coordinación de emergencias, resulta fundamental definir con claridad los actores que interactúan con el sistema. Cada actor representa un perfil de usuario con un conjunto concreto de capacidades, un nivel de acceso determinado y unas responsabilidades asociadas a la gestión de avisos, emergencias, recursos y asignaciones.

En este proyecto se han identificado los siguientes actores:

- **Usuario no autenticado (ACT01):** puede consultar el mapa general y crear un aviso de emergencia sin necesidad de iniciar sesión.
- **Usuario registrado (ACT02):** además de las funciones básicas, puede consultar sus propios avisos y acceder al mapa y a las notificaciones personales.
- **Miembro de equipo (ACT03):** dispone de acceso a la información operativa de su equipo, a las emergencias activas y a las asignaciones asociadas.
- **Responsable de equipo o gestor de recursos (ACT04):** puede gestionar equipos y vehículos, asignar y retirar recursos, y operar sobre emergencias activas dentro de su ámbito de responsabilidad.
- **Coordinador (ACT05):** asume el mayor nivel de permisos y gestiona usuarios, organizaciones, avisos, emergencias, asignaciones y reglas de recomendación.

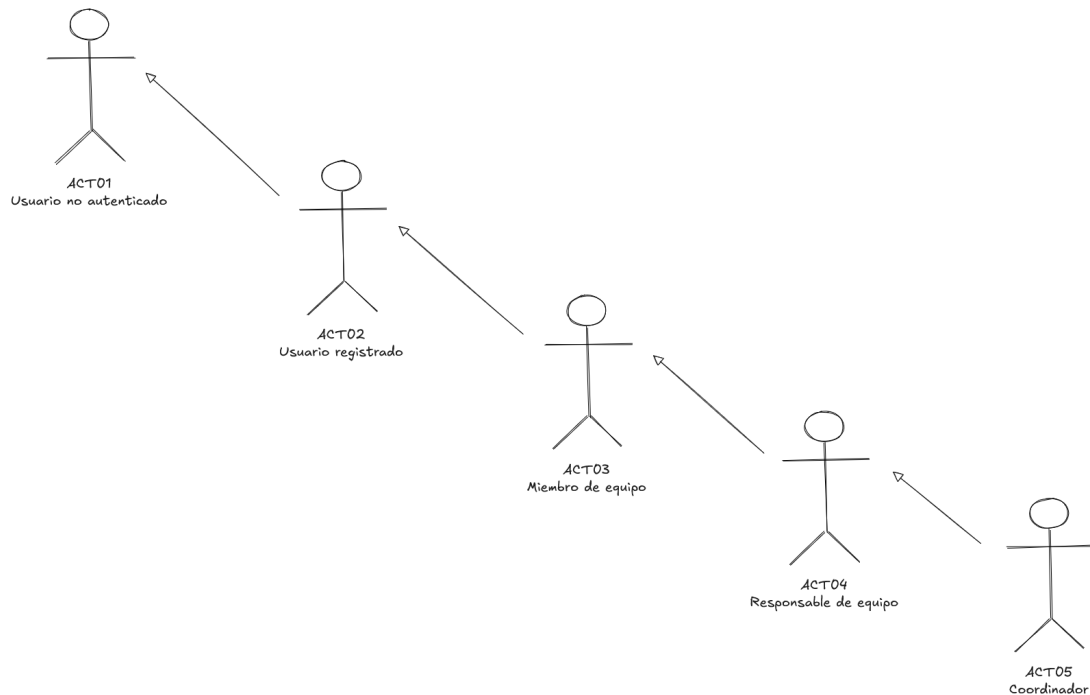


Figura 7.1: Diagrama de la jerarquía de actores del sistema

Además, los permisos siguen una jerarquía acumulativa, como se muestra en la figura 7.1. Esto significa que cada actor superior hereda las capacidades del nivel anterior y añade nuevas funcionalidades. De este modo, el usuario no autenticado posee las capacidades más limitadas, entre ellas se encuentra la visualización del mapa de emergencias y la creación de avisos, mientras que el coordinador, a mayores de las funcionalidades de los roles anteriores cuenta con la capacidad de gestionar usuarios, organizaciones, emergencias y asignaciones, así como de configurar las reglas de recomendación para la asignación automática de recursos. Esta estructura jerárquica facilita la gestión de permisos y asegura que cada usuario tenga acceso únicamente a las funciones necesarias para su rol específico dentro del sistema.

7.3 Casos de uso

Aspectos a considerar

- Agrupar casos de uso (servirá de base para agruparlos en servicios posteriormente en la sección de diseño).
- Definición concreta /detallada de cada caso de uso (precondiciones ...)

7.4 Modelo de casos de uso

Aspectos a considerar

- Diagrama de casos de uso incluyendo actores y relaciones entre casos

7.5 Prototipado de interfaz

Es interesante hacer el esfuerzo en este momento de prototipar las principales pantallas de la interfaz (y añadirlas en la memoria), como parte del análisis de requisitos, así como de aspectos de usabilidad de las mismas.

X^{xxx,...}

8.1 Introducción y objetivos

8.2 Resumen de patrones usados

8.3 Arquitectura general

Aspectos a considerar

- Arquitectura aplicación SPA
- División en subsistemas: En una aplicación SPA al menos backend y frontend pero podría haber más si se decide implementar utilidades e.g. como módulos independientes (e.g. como proyectos maven / jars independientes)

8.4 Subsistema Backend

8.4.1 Objetivos

8.4.2 Arquitectura

Aspectos a considerar

- Diagrama de paquetes general del subsistema (figura y descripción de la misma)

8.4.3 Modelo del dominio

Diagrama de entidades

Aspectos a considerar

- Diagrama de clases persistentes de la aplicación
- Para cada indicar, tanto de su semántica como de sus atributos.
- Justificar decisiones de diseño (¿desnormalizaciones?, métodos de lógica - Domain Model Pattern - ?)

Modelo de datos

Aspectos a considerar

- Diagrama entidad-relación
- Diccionario de Datos (en su ausencia - por temas de espacio bastante razonable - compensable con deficiencia clara en diagrama de entidades).

Consideraciones adicionales de modelado de datos

Aspectos a considerar

- Claves primarias autogeneradas
- Anotaciones para relaciones entre entidades
- Optimistic Locking y @Version en Hibernate
- Relaciones LAZY/EAGER
- Optimizaciones hibernate en navegación entre entidades
- Optimizaciones hibernate - cache primer nivel

8.4.4 Capa de acceso a datos

Aspectos a considerar

- Arquitectura de un DAO
- Aspectos más relevantes de los DAOs implementados

8.4.5 Capa de lógica de negocio

Aspectos a considerar

- Arquitectura de servicios del modelo: Interfaces + clases asociadas
- Incluir descripción de métodos (en algunos casos podrían comentarse de forma agregada algunas operaciones por ejemplo gestión de usuarios (añadir/eliminar/modificar) en lugar de comentar de uno en uno.
- Incluir descripción de algoritmos en los casos que sea necesario. Podría incluirse un diagrama de secuencia para complementar algún algoritmo.

8.4.6 Capa servicios

Aspectos a considerar

- Diseño controladores
- API REST (intentar que sea REST-full cuando haya un mapping directo ... overloading del POST en otro caso
- Gestión de errores y aspectos de internacionalización de mensajes de error
- Autenticación y aspectos de control de acceso a los diferentes endpoints (SecurityConfig). Soporte de redirección tras autenticación, si implementado.

8.4.7 Otros?

Aspectos a resaltar

- Integración con otros sistemas o librerías a nivel de backend, e.g. pago con PayPal.
- Tb se podrían destacar aspectos como chats con websockets o algoritmos. Importante dejar claro todo lo que se comente en "objetivos" y "aportaciones" principales del proyecto.

8.5 Subsistema Frontend

8.5.1 Objetivos

8.5.2 Arquitectura

Aspectos a considerar

- Arquitectura de capas: al menos capa de acceso al servicio y capa de componentes de interfaz

8.5.3 Capa de acceso al servicio

8.5.4 Capa de componentes de interfaz

8.5.5 Otros?

Aspectos a considerar

- Internacionalización de mensajes de texto
- Integración con otros sistemas / librerías

Implementación

X^{xxx,...}

9.1 Software requerido

Pre-requisitos software para poder compilar el código fuente. Respecto a la base de datos, puede referenciarse al apéndice de instrucciones de instalación, a la parte de instalación y configuración de la base de datos.

9.2 Estructura

Estructura de carpetas del código fuente. Si se utiliza maven, básicamente indicar estructura estándar de maven + casos especiales como localización de fichero de creación de datos ...

9.3 Instrucciones de compilación

Instrucciones de cómo compilar el código fuente. Si se usa maven, básicamente comentar las fases principales a usar de maven.

Comentar también cómo ejecutar en desarrollo.

Capítulo 10

Pruebas

X^{xxx,...}

10.1 Introducción

Aspectos a considerar

- Importancia de las pruebas, tipos de pruebas realizadas en este proyecto y metodología de las pruebas.

10.2 Pruebas unitarias

10.3 Pruebas de integración

Aspectos a considerar

- Importante indicar si son automatizadas o manuales.
- En el caso de pruebas del modelo, casi seguro que serán automatizadas. Necesario indicar la idea de las pruebas, repetibles para detectar posibles problemas de regresión en el futuro e independientes unas de otras: cada prueba crea lo necesario para validar el caso de uso, invoca el caso de uso a probar y valida los efectos asociados a la invocación del caso de uso y eliminar todos los datos generados. Importante validar diferentes escenarios de éxito y fallo posibles para cada caso de uso.
- En las pruebas de integración, Recomendable incluir el nivel de cobertura de las mismas(existen plugins que lo calculan de forma sencilla).
- En el caso de pruebas de capa web, si no se han automatizado será necesario describir el plan de pruebas.

10.4 ...

Conclusiones y futuras líneas de trabajo

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras, (no caso de [TFM](#), carácter profesional del mesmo)...

11.1 Conclusiones

11.2 Aprendizajes

11.3 Futuras líneas de trabajo

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

A.1 Instalación del Software

A.2 Manual de usuario

Introducción al layout general de la aplicación, y subapartados para cada una de las secciones principales, para que sea más sencillo de consultar / comprender.

Lista de acrónimos

API Application Programming Interface. [16](#), [17](#)

AXEGA Axencia Galega de Emerxencias (112 Galicia). [1](#), [2](#)

Compose Jetpack Compose. [7](#)

CSS Cascading Style Sheet. [5](#)

FCM Firebase Cloud Messaging. [3](#), [7](#), [14](#), [21](#), [22](#)

FMS FirebaseMessagingService. [7](#)

HTML HyperText Markup Language. [5](#)

HTTP Hypertext Transfer Protocol. [6](#)

HTTPS Hypertext Transfer Protocol Secure. [6](#)

IDE Integrated Development Environment. [6](#)

JavaScript JavaScript. [5](#)

JDBC Java Database Connectivity. [5](#)

JPA Java Persistence API. [5](#), [16](#)

JSON JavaScript Object Notation. [5](#)

JSONB JavaScript Object Notation Binary. [21](#)

JSX JavaScript XML. [5](#)

JUnit JUnit. [6](#)

LaTeX LaTeX. 5

Material Material Design. 7

PLADIGA Plan de Prevención y Defensa Contra los Incendios Forestales de Galicia. 2

React React. 5, 6

Redux Redux. 6

REST Representational State Transfer. 6

RTK Query Redux Toolkit Query. 6

Spring Spring. 5

Spring Boot Spring Boot. 5

Spring Data Spring Data. 5

SQL Structured Query Language. 5

Testcontainers Testcontainers. 7

TFG Trabajo Fin de Grado. 1–3, 8, 11–14, 16, 17, 25, 26

TFM Trabajo Fin de Máster. 1–3, 8, 13–17, 20, 24, 25, 36

Bibliografía

- [1] M. para la Transición Ecológica y el Reto Demográfico (MITECO), “Estadística general de incendios forestales (egif),” 2025, consultado el 2026-05-20. [En línea]. Disponible en: <https://www.miteco.gob.es/es/biodiversidad/temas/incendios-forestales/estadisticas-datos.html>
- [2] A. G. de Emerxencias (AXEGA), “O 112 galicia xestionou unha decena de incidencias relacionadas coa chuvia en ourense, a rúa e o carballiño,” 2026, consultado el 2026-05-20. [En línea]. Disponible en: <https://www.axega112.gal/gl/content/o-112-galicia-xestionou-unha-decena-de-incidencias-relacionadas-coa-chuvia-en-ourense-rua-e>
- [3] —, “Alerta laranxa por chuvias intensas no noroeste de ourense,” 2026, consultado el 2026-05-20. [En línea]. Disponible en: https://www.axega112.gal/gl/alertas_activas
- [4] —, “Rexistrado un sismo de magnitude 2.6 con epicentro en ponteceso,” 2026, consultado el 2026-05-20. [En línea]. Disponible en: <https://www.axega112.gal/gl/content/rexistrado-un-sismo-de-magnitude-26-con-epicentro-en-ponteceso>
- [5] A. García Vázquez, “Aplicación para coordinación de equipos para la prevención y extinción de incendios forestales,” Memoria de TFG, Universidade da Coruña, 2023. [En línea]. Disponible en: <https://ruc.udc.es/entities/publication/04b8901f-0960-4d07-8cce-9a2ecfcfb471>
- [6] A. G. de Emerxencias (AXEGA), “Portal oficial de axega e 112 galicia,” 2026, consultado el 2026-05-20. [En línea]. Disponible en: <https://www.axega112.gal>
- [7] X. e. D. X. d. G. Consellería de Presidencia, “Plans de emerxencia de galicia,” 2026, consultado el 2026-05-20. [En línea]. Disponible en: <https://cpxt.xunta.gal/plans-de-emerxencia>
- [8] C. do Medio Rural e Desenvolvemento Sostible. Xunta de Galicia, “Pladiga: Plan de prevención e defensa contra os incendios forestais (pladiga),” 2026, consultado el

- 2026-05-20. [En línea]. Disponible en: <https://mediorural.xunta.gal/es/temas/defensa-monte/pladiga-2026>
- [9] J. del Estado, “Ley 17/2015, de 9 de julio, del sistema nacional de protección civil,” 2015, consultado el 2026-05-20. [En línea]. Disponible en: <https://www.boe.es/eli/es/l/2015/07/09/17/con>
- [10] M. del Interior, “Real decreto 524/2023, de 20 de junio, por el que se aprueba la norma básica de protección civil,” 2023, consultado el 2026-05-20. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2023/06/20/524/con>
- [11] Oracle, “Java,” <https://www.oracle.com/java/>, consultado el 2026-05-20.
- [12] W3Schools, “Sql,” <https://www.w3schools.com/sql/>, consultado el 2026-05-20.
- [13] M. W. Docs, “Html y css,” https://developer.mozilla.org/es/docs/Learn/Getting_started_with_the_web/HTML_basics, consultado el 2026-05-20.
- [14] Mozilla, “Javascript,” <https://developer.mozilla.org/es/docs/Web/JavaScript>, consultado el 2026-05-20.
- [15] “Jsx (javascript xml),” <https://reactjs.org/docs/introducing-jsx.html>, accedido el 2026-05-20.
- [16] “Latex,” <https://www.latex-project.org/help/documentation/>, accedido el 2026-05-20.
- [17] “Json (javascript object notation),” <https://www.json.org/>, accedido el 2026-05-20.
- [18] Spring, “Spring,” <https://spring.io/>, accedido el 2026-05-20.
- [19] —, “Spring boot,” <https://spring.io/projects/spring-boot>, consultado el 2026-05-20.
- [20] —, “Spring data,” <https://spring.io/projects/spring-data>, consultado el 2026-05-20.
- [21] Oracle, “Java persistence api,” <https://docs.oracle.com/javaee/6/tutorial/doc/bnbpz.html>, consultado el 2026-05-20.
- [22] Hibernate, “Hibernate orm,” <https://hibernate.org/orm/>, consultado el 2026-05-20.
- [23] JDBC, “Jdbc,” <https://www.ibm.com/docs/es/developer-for-zos/9.5.1?topic=support-what-is-jdbc>, consultado el 2026-05-20.
- [24] Facebook, “React,” <https://reactjs.org/>, consultado el 2026-05-20.
- [25] Redux, “Redux,” <https://redux.js.org/>, consultado el 2026-05-20.

- [26] R. Toolkit, “Redux toolkit query,” <https://redux-toolkit.js.org/rtk-query/overview>, consultado el 2026-05-20.
- [27] Material-UI, “Material-ui,” <https://material-ui.com/>, consultado el 2026-05-20.
- [28] O. W. Map, “Open weather map,” <https://openweathermap.org/>, consultado el 2026-05-20.
- [29] M. Community, “Maplibre,” <https://maplibre.org/>, accedido el 2026-05-20.
- [30] JUnit.org, “JUnit,” <https://junit.org/junit5/>, consultado el 2026-05-20.
- [31] Mozilla, “Http,” <https://developer.mozilla.org/es/docs/Web/HTTP>, consultado el 2026-05-20.
- [32] REST, “Rest,” <https://restfulapi.net/>, accedido el 2026-05-20.
- [33] JetBrains, “IntelliJ,” <https://www.jetbrains.com/idea/>, consultado el 2026-05-20.
- [34] Postman, “Postman,” <https://www.postman.com/>, consultado el 2026-05-20.
- [35] Microsoft, “Vscode,” <https://code.visualstudio.com/>, consultado el 2026-05-20.
- [36] DBeaver, “Dbeaver,” <https://dbeaver.io/>, consultado el 2026-05-20.
- [37] Git, “Git,” <https://git-scm.com/>, consultado el 2026-05-20.
- [38] “Apache maven,” <https://maven.apache.org/>, Apache Software Foundation, accedido el 2026-05-20.
- [39] Facebook, “Create react app,” <https://create-react-app.dev/>, consultado el 2026-05-20.
- [40] P. G. D. Group, “Postgresql,” <https://www.postgresql.org/>, consultado el 2026-05-20.
- [41] P. D. Team, “Postgis,” <https://postgis.net/>, consultado el 2026-05-20.
- [42] T. A. S. Foundation, “Apache tomcat,” <https://tomcat.apache.org/>, consultado el 2026-05-20.
- [43] O. S. Community, “Webpack,” <https://webpack.js.org/>, consultado el 2026-05-20.
- [44] Serve, “Serve,” <https://www.npmjs.com/package/serve?activeTab=readme>, consultado el 2026-05-20.
- [45] I. Docker, “Docker,” <https://www.docker.com/>, consultado el 2026-05-20.
- [46] Liquibase.org, “Liquibase,” <https://www.liquibase.org/>, consultado el 2026-05-20.

- [47] H. D. Engine, "H2 database engine," <http://www.h2database.com/html/main.html>, consultado el 2026-05-20.
- [48] Testcontainers, "Testcontainers," <https://testcontainers.com/>, consultado el 2026-05-20.
- [49] R. M. GL, "React map gl," <https://visgl.github.io/react-map-gl/>, consultado el 2026-05-20.
- [50] Mapbox, "Mapbox," <https://www.mapbox.com/>, consultado el 2026-05-20.
- [51] JetBrains, "Kotlin programming language," <https://kotlinlang.org/>, consultado el 2026-05-20.
- [52] G. Inc., "Gradle build tool," <https://gradle.org/>, consultado el 2026-05-20.
- [53] A. Developers, "Jetpack compose," <https://developer.android.com/jetpack/compose>, consultado el 2026-05-20.
- [54] Google, "Material design," <https://material.io/>, consultado el 2026-05-20.
- [55] P. EmergencyApp, "HttpClient (proyecto)," <https://github.com/your/repo>, consultado el 2026-05-20.
- [56] G. Firebase, "Firebase cloud messaging (fcm)," <https://firebase.google.com/docs/cloud-messaging>, consultado el 2026-05-20.
- [57] A. D. . Firebase, "Firebasemessaging (android)," <https://firebase.google.com/docs/cloud-messaging/android/receive>, consultado el 2026-05-20.
- [58] Atlassian, "Jira software," <https://www.atlassian.com/software/jira>, consultado el 2026-05-20.
- [59] I. Asana, "Asana," <https://asana.com/>, consultado el 2026-05-20.
- [60] G. F. Watch, "Global forest watch fires," <https://www.globalforestwatch.org/>, consultado el 2026-05-20.
- [61] U. W. F. Information, "Inciweb - incident information system," <https://inciweb.nwcg.gov/>, consultado el 2026-05-20.
- [62] Google, "Google maps platform," <https://maps.google.com/>, consultado el 2026-05-20.
- [63] Microsoft, "Microsoft teams," <https://www.microsoft.com/en/microsoft-teams/group-chat-software>, consultado el 2026-05-20.
- [64] O. S.A., "Odoo," <https://www.odoo.com/>, consultado el 2026-05-20.

- [65] PCAM, “Pcam gestión,” <https://www.pcamexample.org/>, consultado el 2026-05-20.
- [66] X. de Galicia, “Xeocode lite,” <https://www.xunta.gal/>, herramienta interna y privada de uso operativo.
- [67] L. V. Municipal, “Línea verde municipal - plataforma de participación ciudadana y gestión de incidencias,” <https://www.lineaverdemunicipal.info/>, consultado el 2026-05-20.
- [68] NavegaGPS, “S.o.s emergencias,” <https://play.google.com/store/apps/details?id=navegagps.emergenciasnavegagps&hl=en>, consultado el 2026-05-20.
- [69] X. de Galicia, “La xunta amplía los mecanismos de detección temprana de incendios forestales con una nueva app para la ciudadanía,” <https://www.xunta.gal/es/notas-de-rensa/-/nova/022747/xunta-amplia-los-mecanismos-deteccion-temprana-incendios-forestales-con-una-nueva>, 2026, consultado el 2026-05-20.
- [70] Talent.com, “Salario de un programador en españa,” <https://es.talent.com/salary?job=programador>, consultado el 2026-05-20.
- [71] —, “Salario de un analista programador en españa,” <https://es.talent.com/salary?job=analista+programador>, consultado el 2026-05-20.